ESIGELEC

ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

# Déploiement de solutions IA et cybersécurité

# Cours n°2 : Conteneurisation et infrastructure Cloud IA

**Présenté par : Hichem AMMAR-BOUDJELAL**

**le : 10/11/2026**

# PROGRAMME DU COURS COMPLET

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

## 1 - INTRODUCTION AU DÉPLOIEMENT D'IA ET À LA CYBERSÉCURITÉ



- CYCLE DE VIE D'UN PROJET IA
- CONCEPTS DE MLOPS ET CI/CD
- PREMIERS PRINCIPES DE SÉCURITÉ ET DE CONFORMITÉ
- TP N°1 : ÉTUDE DE CAS

## 2 - CONTENEURISATION ET INFRASTRUCTURE CLOUD



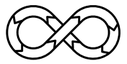
- DOCKER, IMAGES, CONTENEURS, REGISTRES
- KUBERNETES (GKE, KAPSULE, AKS, EKS)
- DÉPLOIEMENT D'UN SERVICE D'INFÉRENCE IA
- SCALABILITÉ HORIZONTALE ET VERTICALE

## 3 - SCALABILITÉ, PERFORMANCE ET MONITORING



- ARCHITECTURE SCALABLE ET TOLÉRANTE AUX PANNES
- AUTOSCALING, LOAD BALANCING ET TESTS DE CHARGE
- OBSERVABILITÉ : LOGS, MÉTRIQUES, TRACES
- MONITORING AVEC PROMETHEUS, GRAFANA, CLOUD MONITORING

## 4 - MLOPS ET PIPELINES D'INTÉGRATION CONTINUE



- CI/CD POUR MODÈLES IA
- VERSIONING DES MODÈLES ET JEUX DE DONNÉES
- GITHUB ACTIONS, CLOUD BUILD, VERTEX PIPELINES
- TESTS D'INTÉGRATION ET VALIDATION AUTOMATIQUE

## 5 - SÉCURITÉ DES API ET GESTION DES IDENTITÉS



- AUTHENTIFICATION (OAUTH2, JWT)
- AUTORISATION (IAM, RÔLES, CLÉS D'ACCÈS)
- CHIFFREMENT, SIGNATURES ET LOGS D'AUDIT
- VULNÉRABILITÉS TYPIQUES DES API IA

## 6 - ATTAQUES ET DÉFENSES IA



- ATTAQUES ADVERSARIALES ET DATA POISONING
- EXTRACTION ET INVERSION DE MODÈLES
- FUZZING ET INJECTION D'INFÉRENCES
- BLUE TEAM VS RED TEAM : STRATÉGIES DÉFENSIVES

## 7 - RGPD, IA RESPONSABLE ET ÉTHIQUE




- PRINCIPES RGPD : CONSENTEMENT, MINIMISATION, DROIT À L'OUBLI
- OUTILS D'ANONYMISATION ET DÉTECTION DE PII (NER)
- TRANSPARENCE, EXPLICABILITÉ ET TRAÇABILITÉ
- AUDITABILITÉ ET DOCUMENTATION IA

## 8 - GOUVERNANCE, DEVSECOPS ET AUDIT



- INTÉGRATION SÉCURITÉ + CONFORMITÉ DANS LE PIPELINE CI/CD
- RÔLES ET RESPONSABILITÉS (DEV, SEC, OPS, DPO, DATA)
- OUTILS DE SCAN (TRIVY, DEPENDABOT, SONARCLOUD)
- GOUVERNANCE DES MODÈLES, DÉRIVE ET SUPERVISION LONG TERME

# 1 - INTRODUCTION À L'INFRASTRUCTURE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES







**QUELLE DIFFÉRENCE ENTRE CES 3 LOGOS ?**

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE



**MODEL AS A SERVICE**

**(MaaS)**



**LOCAL**

**(On-premise)**



**INFRASTRUCTURE AS A SERVICE**

**(IaaS)**

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

| Type | Description | Exemples typiques |
|---|---|---|
| **MaaS** | Déploiement complet du modèle sur plateforme managé | Vertex AI, AWS SageMaker, Azure ML, Hugging Face Inference, OpenAI, Mistral, etc... |
| **On-premise** | Déploiement sur serveurs internes ou postes GPU | Docker, Kubernetes local (K3s, MiniKube), Ollama, etc... |
| **IaaS** | Location d'infrastructure matérielle (VM, stockage, GPU) | GCP Compute Engine, AWS EC2, Azure VM, Scaleway Instances, etc... |

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

**COMMENT CHOISIR ?**

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

## COMMENT CHOISIR ?

- **BUDGET**
- **TEMPS**
- **USE-CASE**
- **CONTRAINTES**
  - **RÉGLEMENTAIRES**
  - **SCALABILITÉ**

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

## EN RÉSUMÉ :

- **EN LOCAL : VOUS MAÎTRISEZ TOUT…MAIS VOUS DEVEZ TOUT MAINTENIR.**
- **EN IAAS : VOUS LOUEZ LA PUISSANCE DE CALCUL.**
- **EN MAAS : VOUS DÉLÉGUEZ 100%, JUSQU'AU DÉPLOIEMENT DU MODÈLE.**

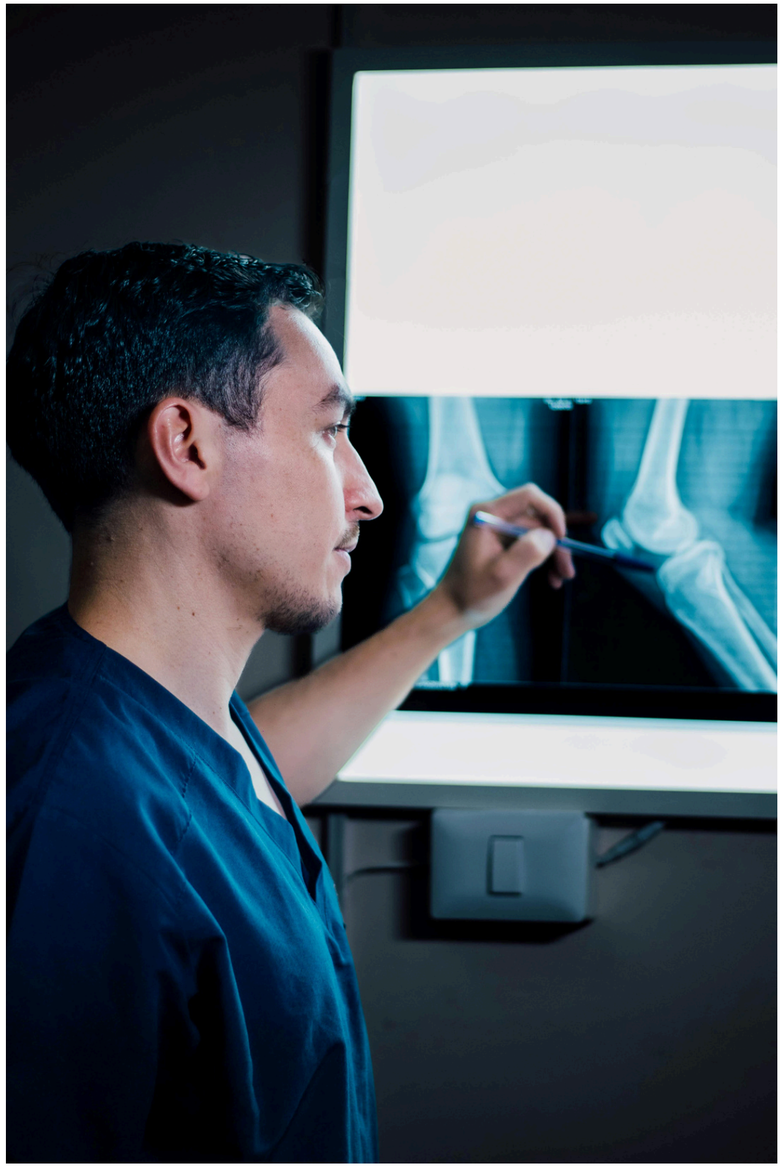
# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

| Type | Description | Usage privilégié |
|---|---|---|
| **MaaS** | Déploiement complet du modèle sur plateforme managé | IA générative, maintenance simplifiée |
| **On-premise** | Déploiement sur serveurs internes ou postes GPU | Prototypage, données sensibles, tests rapides |
| **IaaS** | Location d'infrastructure matérielle (VM, stockage, GPU) | IA prédictive en production, modèles spécifiques |

# 1 - LES 3 GRANDES CATÉGORIES D'INFRASTRUCTURE

| Type d'IA | Exemple | Unité de coût | Facturation typique |
|---|---|---|---|
| IA prédictive | Classification, régression, détection d'image | Temps d'exécution (CPU/GPU) | 3€/heure de calcul |
| IA générative | Chatbot, résumé de texte, génération d'image | Tokens (entrée + sortie) | 0,20€/1M tokens |
| Stockage / données | Base vectorielle ou classique, logs, datasets | Volume (Go/mois) | 0,015€/Go (S3)<br>0,10€/Go (block storage) |
| Réseau / API | Appels HTTP, latence | Requêtes/sortie | 0,15€/million requêtes |

# 2 - CHOISISSEZ VOTRE INFRASTRUCTURE



## CAS D'USAGE N°1 : HÔPITAL UNIVERSITAIRE

Contexte :

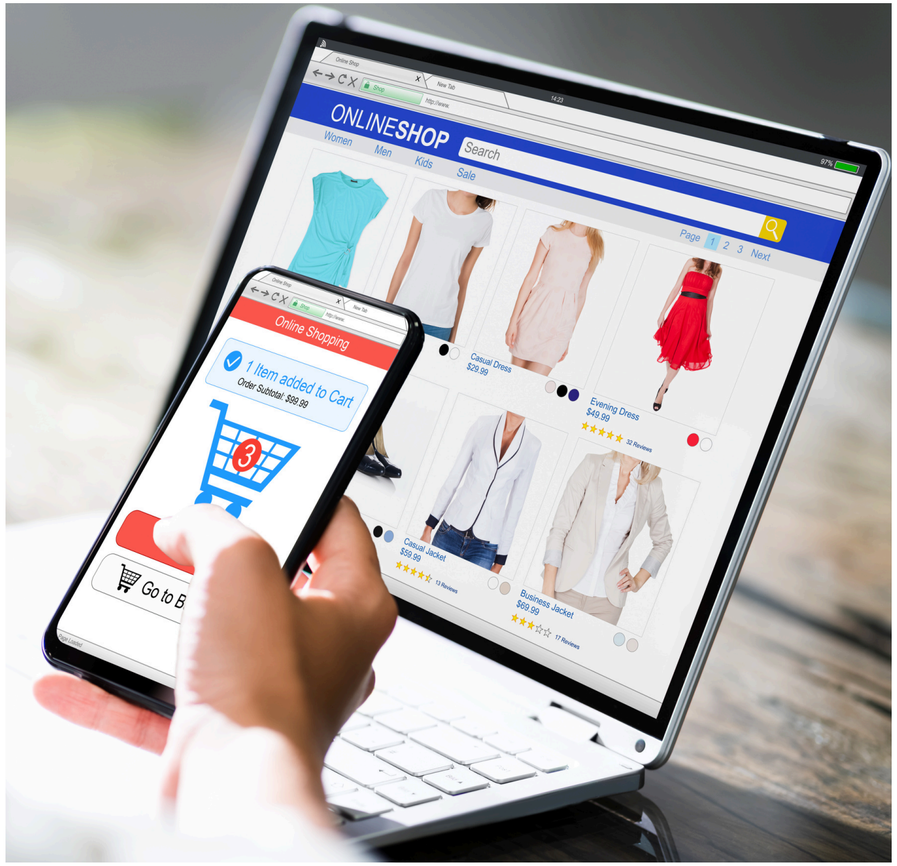
- Modèle CNN de détection de tumeurs sur IRM
- 2000 analyses / jour
- Données HDS, contraintes RGPD strictes
- Besoin : stabilité et traçabilité complète

Question :

- Quelle infrastructure choisir ?
- Pour quel coût mensuel ?

# 2 - CHOISISSEZ VOTRE INFRASTRUCTURE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES




## CAS D'USAGE N°2 : START-UP E-COMMERCE

Contexte :

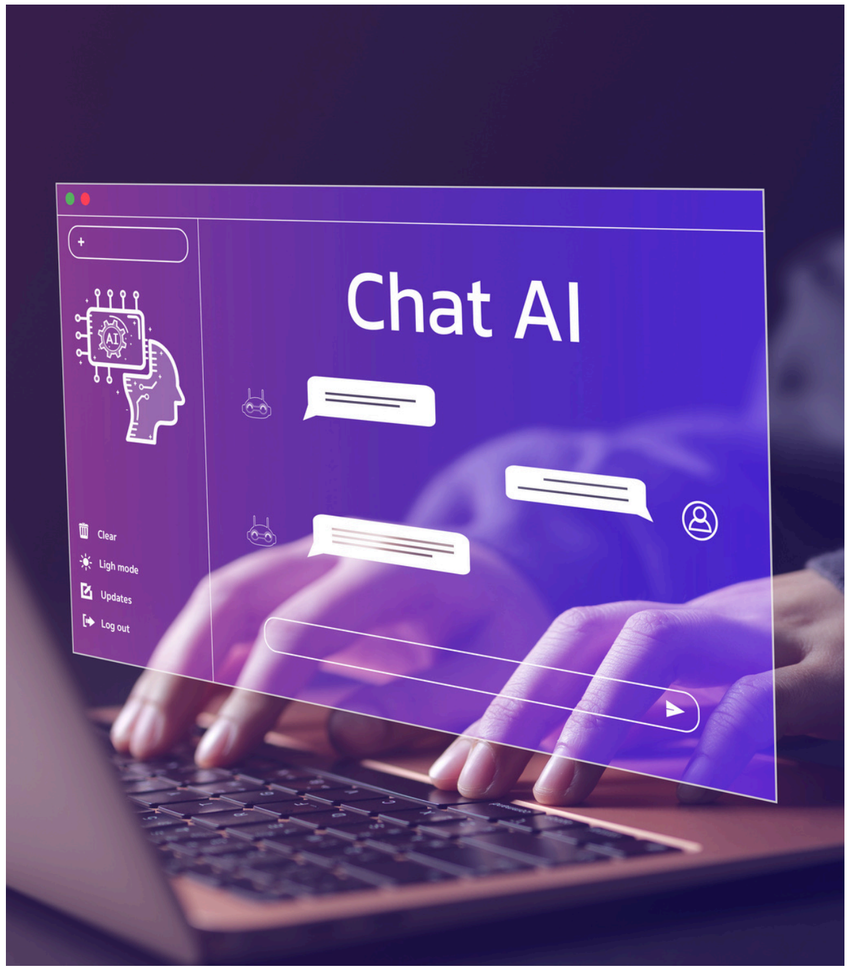
- Modèle de recommandation de produits
- 10 000 requêtes par minute aux heures de pointes
- Objectif : latence < 200 ms
- Besoin de scaler dynamiquement

Question :

- Quelle infrastructure choisir ?
- Pour quel coût mensuel ?

# 2 - CHOISISSEZ VOTRE INFRASTRUCTURE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES




## CAS D'USAGE N°3 : CHATBOT IA GÉNÉRATIVE

Contexte :

- Application conversationnelle avec 50 000 utilisateurs actifs
- Chaque message = **~200** tokens en entrée et **~400** tokens en sortie
- Moyenne : 5 messages / utilisateur / jour

Question :

- Quelle infrastructure choisir ?
- Pour quel coût mensuel ?

# 3 - POURQUOI CONTENEURISER ?

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

**UNE IA EN PRODUCTION DOIT ÊTRE :**

- **REPRODUCTIBLE**
- **DÉPLOYABLE À GRANDE ÉCHELLE**
- **SÉCURISÉE ET OBSERVABLE**

**UTILISER UN CONTENEUR PERMET DE :**

- **PACKAGER UN MODÈLE IA POUR QU'IL S'EXÉCUTE PARTOUT DANS LES MÊMES CONDITIONS**

# 4 - DOCKER VS KUBERNETES





**CRÉER**

**DÉPLOYER**

**EXÉCUTER DES CONTENEURS**



**GÉRER DES CLUSTERS DE CONTENEURS**

**ORGANISE ET AUTOMATISE :**

- **LA MISE EN PLACE**
- **LE DÉPLOIEMENT**
- **LA MISE À L'ÉCHELLE**
- **L'ÉQUILIBRAGE DE CHARGE**
- **MAINTENANCE DES CONTENEURS**

# 5 - DOCKER EN DÉTAILS




- **IMAGE : MODÈLE STATIQUE DE VOTRE ENVIRONNEMENT**
  - **MACHINE VIRTUELLE ALLÉGÉE**
- **CONTENEUR : INSTANCE EN EXÉCUTION D'UNE IMAGE**
- **REGISTRY : DÉPÔT D'IMAGES**
  - **DOCKER HUB, GCP ARTIFACT REGISTRY, ETC...**

# 5 - DOCKER EN DÉTAILS

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES




```
FROM PYTHON:3.10-SLIM
WORKDIR /APP
COPY . /APP
RUN PIP INSTALL -R REQUIREMENTS.TXT
CMD ["UVICORN", "APP:APP", "--HOST", "0.0.0.0", "--PORT", "8080"]
```

**EXEMPLE DE DOCKERFILE SIMPLE**

# 5 - DOCKER EN DÉTAILS



## NOTIONS IMPORTANTES (1/2)

- CRÉER UN FICHIER .DOCKERIGNORE
  - EXCLURE .GIT, DATASETS, NOTEBOOKS, ETC...
- FIXER LES VERSIONS DES DÉPENDANCES
- --NO-CACHE-DIR POUR ÉVITER DE STOCKER LES ARCHIVES
- CRÉER PLUSIEURS WORKERS (GUNICORN)
  - GÉRER PLUSIEURS REQUÊTES EN PARALLÈLE

# 5 - DOCKER EN DÉTAILS



## NOTIONS IMPORTANTES (2/2)

- UTILISATEUR NON-ROOT (RUN USERADD -M APPUSER)
- HEALTHCHECK --INTERVAL=30S --TIMEOUT=3S CMD WGET -QO-HTTP://LOCALHOST:8080/HEALTH
- STOCKER LES SECRETS DANS UN SECRET MANAGER

# 5 - OPTIMISATION ET SÉCURITÉ



**IMAGE PYTHON**

**APERÇU DES IMAGES DISPONIBLES**

3.15.0a1-trixie

3.15.0a1-slim-trixie

3.15.0a1-bookworm

3.15.0A1-slim-bookworm

3.15.0a1-alpine3.22

...

# 5 - OPTIMISATION ET SÉCURITÉ



**IMAGE PYTHON**

**APERÇU DES IMAGES DISPONIBLES**

Image Python complète : entre 900 Mo et 1,3 Go

Image Python slim ou alpine : 400 Mo

**DIFFÉRENCE**

Paquets non-essentiels supprimés

# 5 - OPTIMISATION ET SÉCURITÉ



**IMAGE PYTHON**

**EXÉCUTER UN UTILISATEUR NON-ROOT :**

```
RUN adduser --disabled-password appuser
USER appuser
```

**SCANNER LES VULNÉRABILITÉS**

Trivy

Docker Scan (utilise Trivy)



**PAS DE CLÉS SECRÈTES DANS LE CONTENEUR !**

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

# 5 - OPTIMISATION ET SÉCURITÉ



**EXEMPLE : HACK DE TWITCH EN OCTOBRE 2021**

- Infiltration des dépôts git internes
- 200 Go de code source téléchargé
- Diffusé sur le forum 4chan
- Twitch a été acheté 970 millions $
  - en 2014

**PAS DE CLÉS SECRÈTES DANS LE CONTENEUR !**

# 6 - POURQUOI ORCHESTRER ?



kubernetes



**VS**



kubernetes

**GÈRE 1 SEUL CONTENEUR**

**GÈRE DES CENTAINES OU MILLIERS DE CONTENEURS**

- **DÉPLOIEMENT**
- **SCALING**
- **SANTÉ**
- **MISE À JOUR CONTINUE**

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

# 7 - KUBERNETES EN DÉTAILS





kubernetes

- **POD : UNITÉ D'EXÉCUTION (1 OU PLUSIEURS CONTENEURS)**
- **DEPLOYMENT : STRATÉGIE DE DÉPLOIEMENT**
- **SERVICE : POINT D'ACCÈS RÉSEAU**
- **INGRESS : EXPOSE UN SERVICE SUR INTERNET**

## POURQUOI UTILISER KUBERNETES ?

# 7 - KUBERNETES EN DÉTAILS

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

NETFLIX

- 300 MILLIONS D'ABONNÉS (DÉBUT 2025)
- 80 ALGORITHMES DE RECOMMANDATION
  - DIFFÉRENTS POUR CHAQUE UTILISATEUR
- DES CENTAINES DE MILLIONS DE PRÉDICTIONS / JOUR




**SI J'UTILISE UN CONTENEUR SIMPLE ?**

# 7 - KUBERNETES EN DÉTAILS

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

NETFLIX

- **300 MILLIONS D'ABONNÉS (DÉBUT 2025)**
- **80 ALGORITHMES DE RECOMMANDATION**
  - **DIFFÉRENTS POUR CHAQUE UTILISATEUR**
- **DES CENTAINES DE MILLIONS DE PRÉDICTIONS / JOUR**




**AVEC DES WORKERS ?**

# 7 - KUBERNETES EN DÉTAILS

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

NETFLIX

- 300 MILLIONS D'ABONNÉS (DÉBUT 2025)
- 80 ALGORITHMES DE RECOMMANDATION
  - DIFFÉRENTS POUR CHAQUE UTILISATEUR
- DES CENTAINES DE MILLIONS DE PRÉDICTIONS / JOUR



# 7 - KUBERNETES EN DÉTAILS





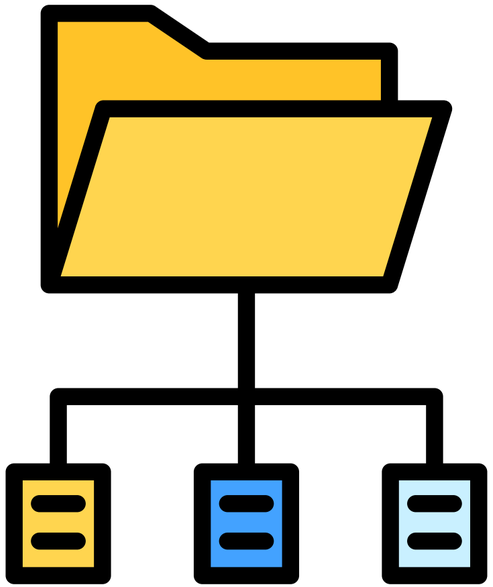
kubernetes

- **RÉPARTIT LA CHARGE**
  - **LOAD BALANCING ?**
- **REDÉMARRE LES PODS EN PANNE**
- **ASSURE QUE LA BONNE VERSION TOURNE PARTOUT**

# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE

| Scalabilité | **Principe** | **Exemple** |
|---|---|---|
| Verticale | Ajouter des ressources à une machine (RAM, CPU, GPU) | Passer de n1-standard-2 à n1-highmem-8 |
| Horizontale | Ajouter des machines / conteneurs | Passer de 2 pods à 10 pods |
| Automatique | Kubernetes ajuste selon la charge | Horizontal Pod Autoscaler |

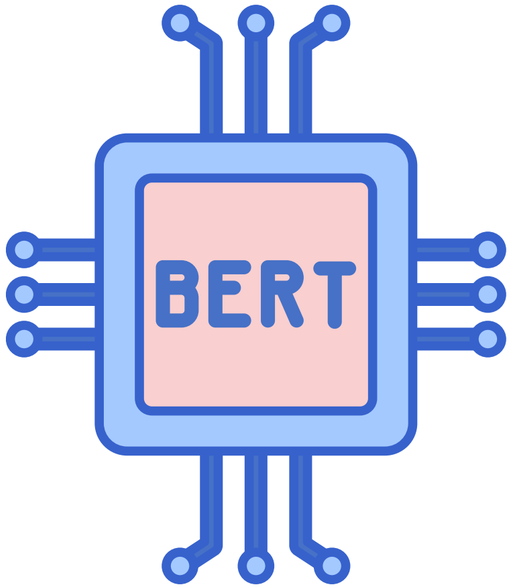
# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE



**EXEMPLE : VOUS DEVEZ DÉPLOYER UNE API DE CLASSIFICATION D'IMAGES POUR 50 000 UTILISATEURS.**

- **QUELLE SCALABILITÉ METTEZ-VOUS EN PLACE ?**

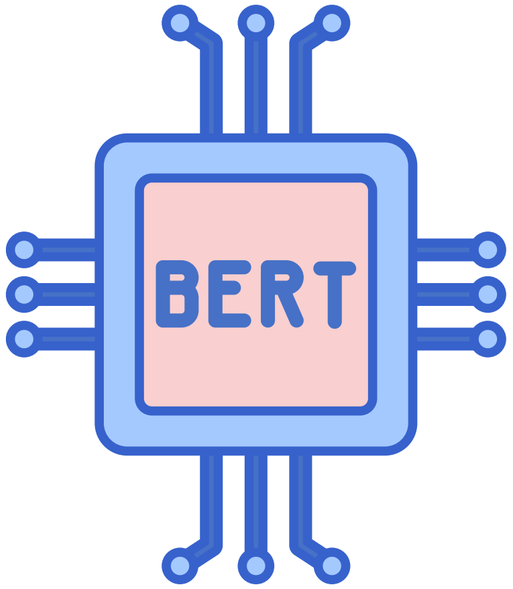
# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE




**EXEMPLE : UN MODÈLE BERT PREND 3 SECONDES À SE CHARGER, MAIS LES CONTENEURS SE RECRÉENT FRÉQUEMMENT.**

- **QUELLE OPTIMISATION PROPOSEZ-VOUS ?**

# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE



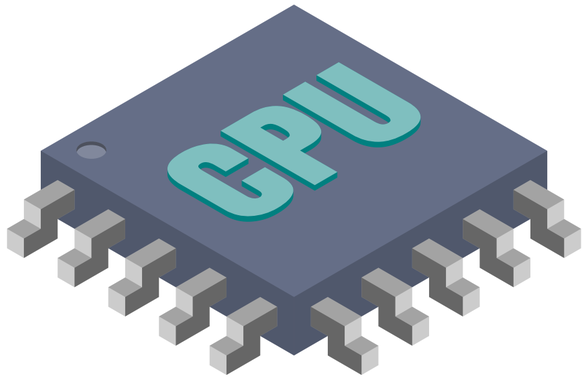

**EXEMPLE : UN MODÈLE BERT PREND 3 SECONDES À SE CHARGER, MAIS LES CONTENEURS SE RECRÉENT FRÉQUEMMENT.**

- **QUELLE OPTIMISATION PROPOSEZ-VOUS ?**

***PRÉCHARGEMENT DANS L'INIT CONTAINER***

***LAZY LOADING + CACHE MÉMOIRE***

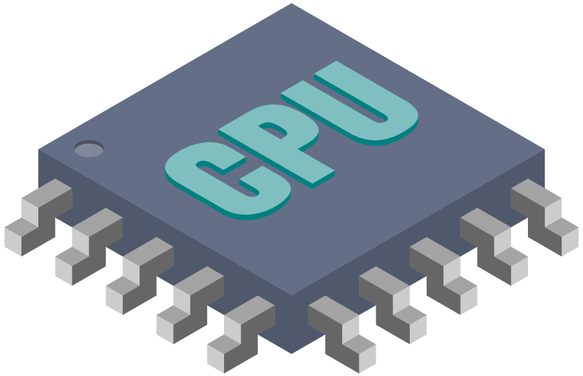
# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE




**EXEMPLE : SUR GKE, VOTRE POD CONSOMME 90% DU CPU.**

- **COMMENT LE SURVEILLEZ-VOUS ?**

# 8 - SCALABILITÉ HORIZONTALE ET VERTICALE




**EXEMPLE : SUR GKE, VOTRE POD CONSOMME 90% DU CPU.**

- **COMMENT LE SURVEILLEZ-VOUS ?**

- ***kubectl top pods***
- ***Prometheus metrics***
- ***Google Cloud Monitoring***

# 9 - CONCLUSION

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES




**STANDARDISATION DES ENVIRONNEMENTS**




**ORCHESTRATION, REDONDANCE, SCALABILITÉ**




**AUTOMATISATION ET SUPERVISION INTÉGRÉES**

# 10 - SERVICE D'INFÉRENCE IA

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

***UN SERVICE D'INFÉRENCE EST UNE APPLICATION QUI EXPOSE UN MODÈLE IA ENTRAÎNÉ SOUS FORME D'API***



## Objectif

**PERMETTRE À D'AUTRES SYSTÈMES (SITE WEB, APP MOBILE, PIPELINE) D'ENVOYER UNE REQUÊTE ET OBTENIR UNE PRÉDICTION**

**EXEMPLES :**

- **/predict → reçoit un texte → renvoie un score de toxicité**
- **/classify → reçoit une image → renvoie une catégorie**

# 10 - SERVICE D'INFÉRENCE IA

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

***UN MODÈLE IA, C'EST JUSTE UN FICHIER .PKL OU .PT***

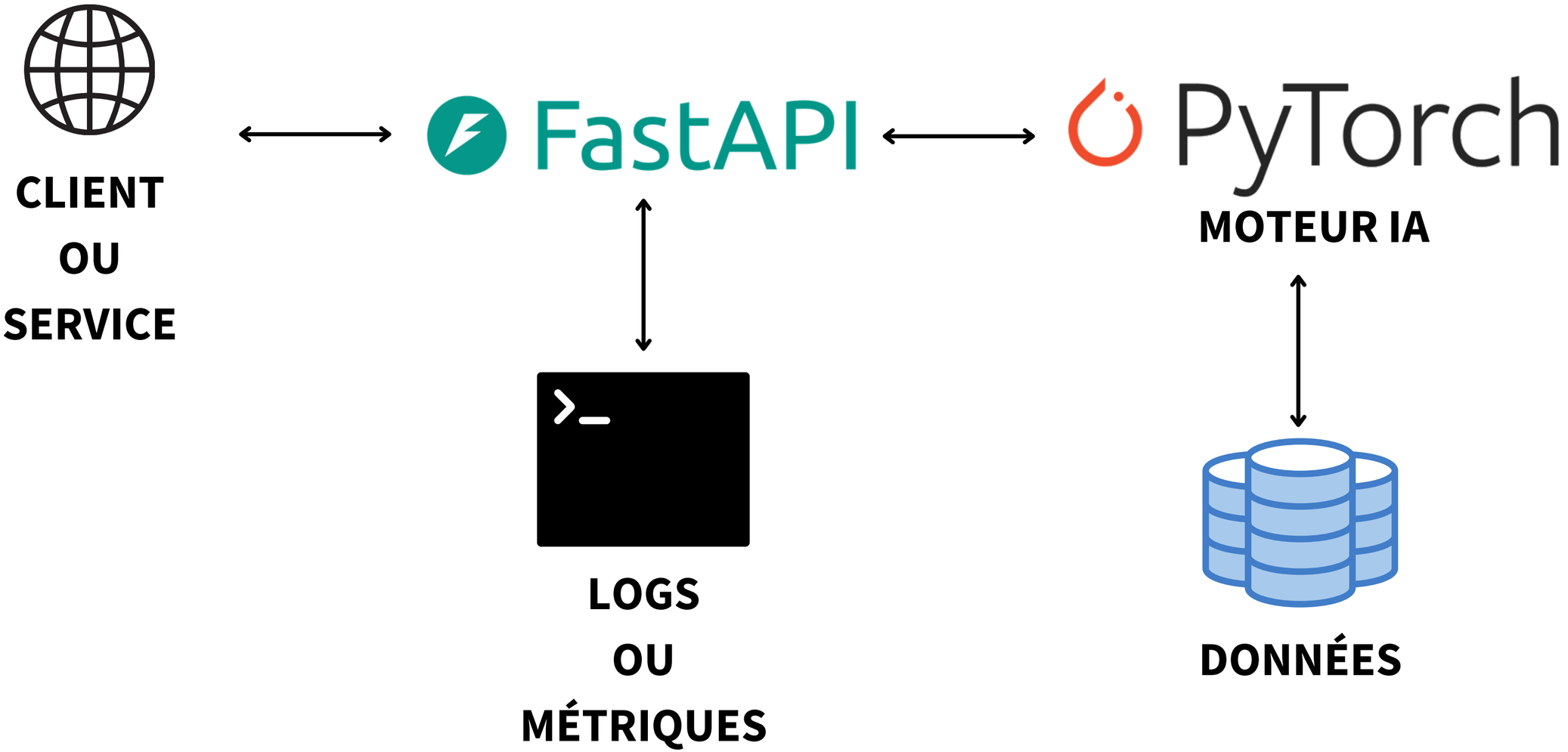
**POUR QU'IL SOIT UTILE, IL FAUT UN SERVICE AUTOUR (UNE API) QUI LE RENDE DISPONIBLE AU MONDE**

**INFÉRENCE LOCALE ?**

**INFÉRENCE SERVICE ?**

# 10 - SERVICE D'INFÉRENCE IA

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES



# 11 - DÉPLOIEMENT

```
from fastapi import FastAPI, Request
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

app = FastAPI()

# Chargement du modèle
model_name = "unitary/toxic-bert"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

@app.post("/predict")
async def predict(request: Request):
    data = await request.json()
    text = data["text"]
    inputs = tokenizer(text, return_tensors="pt", truncation=True)
    outputs = model(**inputs)
    score = torch.sigmoid(outputs.logits).item()
    return {"toxicity_score": round(score, 3)}
```

1. **ON CHARGE UN MODÈLE PRÉ-ENTRAÎNÉ**
   - **TOXIC BERT**
2. **ON REÇOIT UN TEXTE VIA POST**
3. **TOKENISATION**
4. **RENVOIE DU SCORE**



kubernetes

# 11 - DÉPLOIEMENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

```
from fastapi import FastAPI, Request
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

app = FastAPI()

# Chargement du modèle
model_name = "unitary/toxic-bert"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

@app.post("/predict")
async def predict(request: Request):
    data = await request.json()
    text = data["text"]
    inputs = tokenizer(text, return_tensors="pt", truncation=True)
    outputs = model(**inputs)
    score = torch.sigmoid(outputs.logits).item()
    return {"toxicity_score": round(score, 3)}
```

***QUELS PROBLÈMES POTENTIELS EN PRODUCTION ?***

# 11 - DÉPLOIEMENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

```
from fastapi import FastAPI, Request
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

app = FastAPI()

# Chargement du modèle
model_name = "unitary/toxic-bert"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

@app.post("/predict")
async def predict(request: Request):
    data = await request.json()
    text = data["text"]
    inputs = tokenizer(text, return_tensors="pt", truncation=True)
    outputs = model(**inputs)
    score = torch.sigmoid(outputs.logits).item()
    return {"toxicity_score": round(score, 3)}
```

***QUELS PROBLÈMES POTENTIELS EN PRODUCTION ?***

- **TEMPS DE CHARGEMENT DU MODÈLE**
- **GESTION GPU**
- **CHARGE MÉMOIRE**
- **AUTHENTIFICATION**

# 11 - DÉPLOIEMENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: toxicity-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: toxicity-api }
  template:
    metadata:
      labels: { app: toxicity-api }
    spec:
      containers:
      - name: toxicity-api
        image: gcr.io/mon-projet/toxicity-api:1.0.0
        ports: [{ containerPort: 8080 }]
        readinessProbe:
          httpGet: { path: /ready, port: 8080 }
        resources:
          requests: { cpu: "500m", memory: "1Gi" }
          limits: { cpu: "1", memory: "2Gi" }
---
apiVersion: v1
kind: Service
metadata:
  name: toxicity-service
spec:
  type: LoadBalancer
  selector: { app: toxicity-api }
  ports: [{ port: 80, targetPort: 8080 }]
```

**Manifeste K8s :**

- On décrit comment les conteneurs doivent tourner
  - 3 replicas
- On crée un point d'accès réseau stable
- Type : LoadBalancer
  - GKE crée automatiquement une IP publique accessible sur Internet

# 11 - DÉPLOIEMENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

**Étapes du déploiement :**

1. Build → Dockerfile : Création de l'image
2. Push → Déploiement de l'image sur un Registry (GCP, Azure, AWS, Scaleway)
3. Deploy → YAML (GKE, AKS, Kapsule)
4. Expose → Service / LoadBalancer
5. Monitor → Logs, métriques, alertes

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

# MERCI DE VOTRE ATTENTION



**Contact :**
**Hichem AMMAR-BOUDJELAL**
**Mail : hichem@dpliance.com**